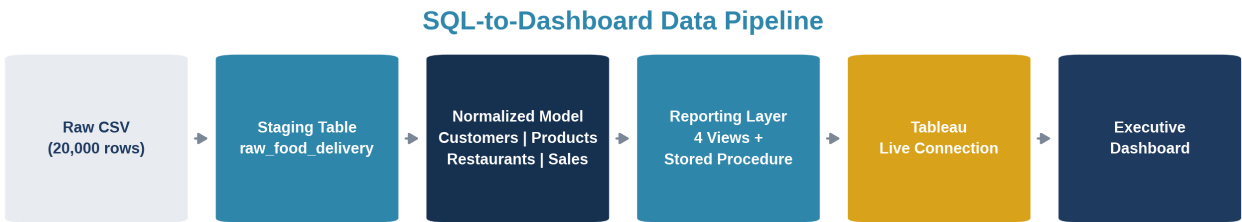


SQL-Driven Business Analytics & Live Tableau Dashboard

A Portfolio Case Study — Food Delivery Business Analytics



The complete SQL-to-dashboard workflow documented in this report

PREPARED BY Subham Kumar	DATASET SCOPE 20,000 Orders · 2024	TOOLS USED MySQL · Tableau Desktop
------------------------------------	--	--

September 2026

Table of Contents

1. Executive Summary	3
2. Business Problem & Objectives	3
3. Dataset & Data Preparation	4
4. SQL Architecture & Data Model	5
5. Advanced SQL Analytics	6
6. SQL Reporting Layer	7
7. Stored Procedure — sp_MonthlySummary	7
8. Tableau Live-Connected Dashboard	8
9. Executive Dashboard — Visual Documentation	8
10. Business Insights	9
11. Performance Optimization	10
12. Technical Challenges & Solutions	10
13. Lessons Learned	10
14. Limitations & Future Improvements	11
15. Final Deliverables & Submission Package	11
16. Conclusion	12

1. Executive Summary

This project builds a production-style business analytics pipeline for a food-delivery operations dataset. The work begins with a raw CSV export of 20,000 orders and ends with a live, filterable executive dashboard in Tableau — everything in between, from table design to window-function analytics to stored procedures, was built and tested as part of this task.

The goal was not simply to “run some SQL” but to reproduce how a small analytics team would actually organize this work: separate the messy source data from a clean relational model, encode the recurring business questions as reusable views, and let the BI layer read from a single trustworthy source rather than a spreadsheet.

Snapshot

20,000 Total Orders	546,702 Total Revenue	27.34 Avg Order Value	2.98 / 5 Avg Satisfaction
-------------------------------	---------------------------------	---------------------------------	-------------------------------------

Live KPI snapshot from the Tableau dashboard — values shift as filters are applied.

What was delivered

- A normalized MySQL schema — Customers, Products, Restaurants, and a Sales fact table — built from a validated staging load of 20,000 rows.
- Window-function analytics: RANK, DENSE_RANK, LAG, LEAD, layered CTEs, and a comparative subquery.
- Four production-style reporting views and one parameterized stored procedure (sp_MonthlySummary).
- Three indexes added and inspected with EXPLAIN to support the dashboard's common query paths.
- A live-connected, multi-filter Tableau executive dashboard covering revenue, customers, products, geography, delivery, and weather/traffic conditions.

2. Business Problem & Objectives

2.1 The problem

Spreadsheet and flat-file analysis works fine for a one-off question, but it breaks down once a business needs repeatable reporting, several dashboards drawing on the same numbers, and logic that has to stay consistent across teams. Every new question tends to spawn a new, slightly different copy of the data — and the copies quietly drift apart. Task 6 exists to remove that drift: move the analytical logic into a relational SQL backend once, and let every downstream report — including the BI layer — query that single source.

2.2 Objectives

#	Objective	Status
1	Design a normalized relational database from the raw export	Complete
2	Separate staging data from reporting-ready entities	Complete
3	Apply window functions and subqueries for business analytics	Complete
4	Create reusable, production-style reporting views	Complete
5	Automate a monthly executive summary via stored procedure	Complete
6	Optimize common query paths with indexes + EXPLAIN	Complete
7	Build a live-connected Tableau executive dashboard	Complete
8	Document the full SQL-to-dashboard workflow	Complete (this report)

3. Dataset & Data Preparation

3.1 Dataset profile

The source file is a single CSV export covering the 2024 calendar year. It was loaded as-is into a staging table before any cleaning or restructuring, so the original values could always be re-checked against the normalized model.

Attribute	Value
Rows	20,000
Columns	31
Unique orders	20,000
Unique customers	20,000
Restaurants	100
Distinct food items	21
Order period	2024 (full calendar year)
Missing values	None observed
Duplicate rows	None observed

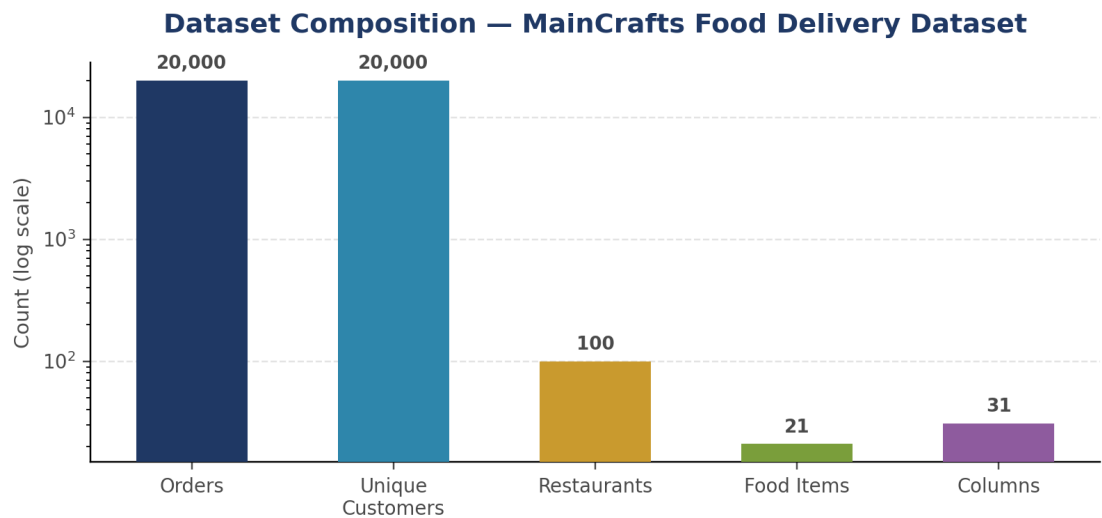


Figure 3.1 — Dataset composition by entity count (log scale, since orders and restaurants sit on very different magnitudes).

3.2 Key fields used in analysis

order_id, customer_id, restaurant_id, food_item, order_time, delivery_time, order_value, location, gender, customer_satisfaction, delivery_delay, traffic_condition, and weather_condition.

3.3 Preparation notes

Boolean-looking True/False fields in the source were kept as VARCHAR(5) at the staging stage rather than cast to a native boolean type immediately — this preserved the exact source representation and made it possible to catch formatting inconsistencies before they propagated into the normalized tables.

3.4 Honest limitations of this dataset

- There is no genuine Profit column in the source data, so order value is reported as revenue — never relabeled as profit.
- Every customer_id in the file is unique, so this dataset cannot support repeat-customer or lifetime-value analysis. That kind of analysis would need a dataset with repeat visits per customer.
- A few Task-6 template concepts (“Sales”, “Products”, “Customers”) were mapped onto the actual food-delivery fields rather than invented from scratch — the mapping is described in Section 4.

4. SQL Architecture & Data Model

The solution follows a staging → normalized database → reporting layer → BI dashboard pattern. Keeping these as distinct layers means a mistake in one layer (say, a bad row in the CSV) never has to be chased through four different places — it is caught and fixed once, at the staging boundary.

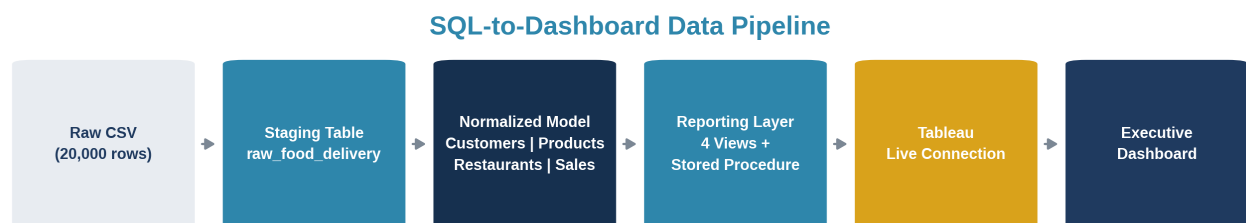


Figure 4.1 — End-to-end data flow from raw CSV to the executive dashboard.

4.1 Tables

Table	Role
raw_food_delivery	Staging table — exact copy of the source CSV, used only for validation and re-load.
Customers	One row per customer: gender, location, and satisfaction score.
Products	One row per distinct food item.
Restaurants	One row per restaurant.
Sales	Transactional fact table — one row per order, linking Customers, Products, and Restaurants.

4.2 Entity-relationship model

Normalized Relational Model (Star-style Schema)

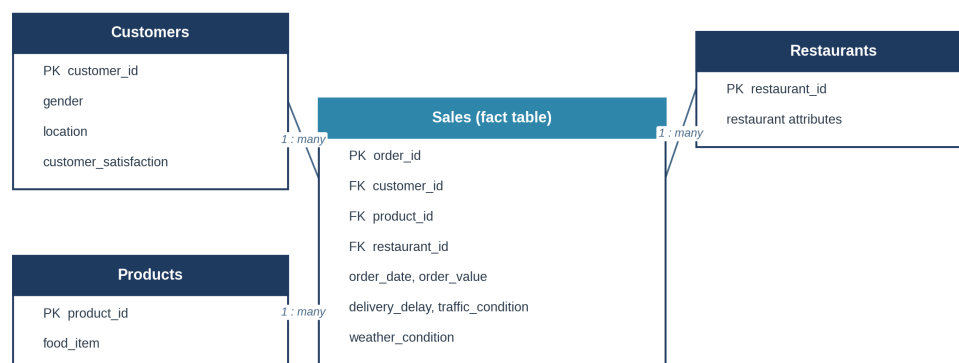


Figure 4.2 — Sales sits at the center as the fact table; Customers, Products, and Restaurants are the surrounding dimension tables, each in a 1-to-many relationship with Sales.

5. Advanced SQL Analytics

Six analytical techniques were used to answer the kinds of questions a business stakeholder actually asks — “who are our best customers,” “which products earn the most,” “are we growing month over month.” Each is shown below with the working query.

5.1 RANK — customer revenue ranking

```
SELECT customer_id, SUM(order_value) AS total_revenue,  
RANK() OVER (ORDER BY SUM(order_value) DESC) AS revenue_rank  
FROM Sales  
GROUP BY customer_id  
ORDER BY revenue_rank;
```

Used to build the Top-10-customer view of the dashboard. RANK() was chosen over ROW_NUMBER() here because tied revenue totals should share the same rank.

5.2 DENSE_RANK — product revenue ranking

```
SELECT product_id, SUM(order_value) AS total_revenue,  
DENSE_RANK() OVER (ORDER BY SUM(order_value) DESC) AS revenue_rank  
FROM Sales  
GROUP BY product_id  
ORDER BY revenue_rank;
```

DENSE_RANK avoids gaps in the ranking sequence when two products tie — important with only 21 distinct food items, where ties are common.

5.3 & 5.4 LAG and LEAD — month-over-month comparison

```
WITH MonthlySales AS (  
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS sales_month,  
  SUM(order_value) AS monthly_revenue  
  FROM Sales  
  GROUP BY DATE_FORMAT(order_date, '%Y-%m')  
)  
SELECT sales_month, monthly_revenue,  
LAG(monthly_revenue) OVER (ORDER BY sales_month) AS previous_month_revenue,  
LEAD(monthly_revenue) OVER (ORDER BY sales_month) AS next_month_revenue  
FROM MonthlySales  
ORDER BY sales_month;
```

LAG pulls the prior month's revenue into the same row for growth comparison; LEAD was tested the same way to expose the following month for forward-looking checks.

5.5 CTE + month-over-month growth

Building on the query above, a second CTE stage takes previous_month_revenue and computes percentage growth: $((\text{monthly_revenue} - \text{previous_month_revenue}) / \text{previous_month_revenue}) \times 100$. Structuring this as two chained CTEs — rather than one dense query — kept the logic readable enough that it could be handed to a non-technical stakeholder for review.

5.6 Subquery — above-average orders

```
SELECT customer_id, order_value, order_date
FROM Sales
WHERE order_value > (SELECT AVG(order_value) FROM Sales)
ORDER BY order_value DESC;
```

A correlated-free scalar subquery, used to isolate the segment of orders worth more attention from a revenue-operations standpoint.

6. SQL Reporting Layer

Rather than let Tableau query the raw Sales table directly for every worksheet, four views were built to pre-shape the most common questions. This keeps the dashboard's calculated fields simpler and means the same business logic (e.g., “what counts as monthly revenue”) is defined once, in SQL, instead of once per worksheet.

View	Purpose
vw_CustomerKPIs	Customer-level totals that support ranking, filtering, and top-customer KPI cards.
vw_MonthlyTrend	Monthly order count, revenue, and average order value — feeds the trend line chart.
vw_ProductPerformance	Product-level order count, revenue, average order value, satisfaction, and freshness measures.
vw_RegionalSummary	Location-level orders, revenue, average order value, satisfaction, and delivery delay.

All four views were created and verified directly inside the Task6_Business_Analytics schema before Tableau was connected, so the dashboard was always reading from tested, working views rather than in-progress SQL.

7. Stored Procedure — sp_MonthlySummary

The one recurring request every stakeholder eventually makes is “just give me last month's numbers.” Rather than re-writing that query by hand each time, it was wrapped in a parameterized stored procedure that takes a reporting month and returns a one-row executive summary.

```
CREATE PROCEDURE sp_MonthlySummary(IN p_report_month VARCHAR(7))
BEGIN
  SELECT p_report_month AS report_month,
  COUNT(s.order_id) AS total_orders,
  ROUND(SUM(s.order_value), 2) AS total_revenue,
  ROUND(AVG(s.order_value), 2) AS average_order_value,
  ROUND(AVG(s.delivery_delay), 2) AS average_delivery_delay,
  ROUND(AVG(c.customer_satisfaction), 2) AS average_customer_satisfaction
  FROM Sales s
  JOIN Customers c ON s.customer_id = c.customer_id
  WHERE DATE_FORMAT(s.order_date, '%Y-%m') = p_report_month;
END;

CALL sp_MonthlySummary('2024-01');
```

The procedure was executed for January 2024 and returned the expected single-row summary, confirming the join and date-formatting logic behave correctly across the full year of data.

8. Tableau Live-Connected Dashboard

Tableau Desktop connects to MySQL through a live connection — not an extract — so the dashboard always reflects the current state of the database rather than a snapshot taken at export time. A unified “Sales+” data source was built by joining Customers, Sales, and Products, so a single filter (say, Location) propagates correctly across every worksheet that uses it.

8.1 Dashboard components

Component	What it shows
KPI cards	Total Orders, Total Revenue, Average Order Value, Customer Satisfaction
Monthly Revenue Trend	Line chart of revenue across the 2024 calendar
Top 10 Customers	Ranked bar chart using the RANK() view
Customer Satisfaction by Gender	Radial / circle-based comparison
India Revenue Map	Geographic view of revenue by location
Product Performance	Revenue and satisfaction by food item
Regional Summary	Orders, revenue, and delivery delay by location
Delivery Performance	Lollipop chart comparing average delay
Traffic & Weather Impact	Heatmap of average delivery delay by condition

8.2 Interactive filters

- Location
- Gender
- Loyalty Program
- Delivery Method (where applicable in the source data)

The dashboard was tested by toggling Location, Gender, and Loyalty Program and confirming every worksheet updated consistently. The India map is intentionally scoped as a location-based geographic view rather than a full route-level delivery map.

9. Executive Dashboard — Visual Documentation

The figure below is an actual screenshot of the finished, live-connected Tableau workbook — not a mock-up. It shows the dashboard in its default (unfiltered) state, with all ten location filters, the full gender/loyalty/delivery-method filter panel, and every one of the nine worksheets described in Section 8.1 visible in a single view.



Figure 9.1 — Executive Business Analytics dashboard, captured directly from Tableau Desktop.

9.1 Visual design choices

- Dark, executive-style background so KPI cards and chart colors carry more visual weight.
- Consistent KPI card styling across the top row for quick scanning.
- A deliberately mixed visual vocabulary — line, radial, map, bar, lollipop, and heatmap — rather than defaulting to bar charts everywhere.
- A single interactive filter panel (Location, Gender, Loyalty Program, Delivery Method) shared across the dashboard.

10. Business Insights

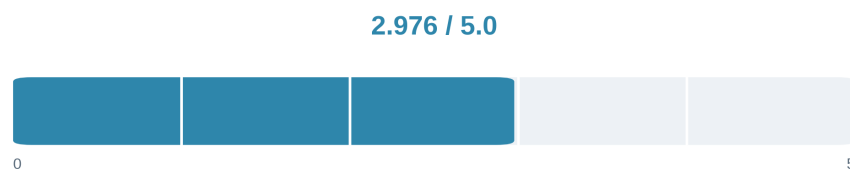


Figure 10.1 — Average customer satisfaction sits at 2.976 out of 5 across all 20,000 orders.

- The dataset's 20,000 orders support month-level revenue analysis across the full 2024 calendar year.
- Revenue can be sliced by location, food item, and customer from a single SQL-backed model — no re-querying the raw CSV needed.
- Customer ranking surfaces the highest-value order records, though — as noted in Section 3.4 — the one-order-per-customer structure of this dataset limits true repeat-customer interpretation.
- Product performance highlights which of the 21 food items contribute the most order value.
- Regional summaries combine revenue with satisfaction and delivery-delay indicators in one view, so a location that looks strong on revenue but weak on delay is easy to spot.

- Delivery Performance compares average delay across delivery methods.
- Traffic & Weather Impact gives a matrix view of how average delivery delay shifts across operational conditions.
- All of the above remain filterable live by location, gender, and loyalty-program status.

KPI snapshot (unfiltered)

20,000 Total Orders	546,702 Total Revenue	27.34 Avg Order Value	2.976 / 5 Avg Satisfaction
-------------------------------	---------------------------------	---------------------------------	--------------------------------------

These four figures move whenever a filter is applied — they are reported here as the unfiltered, whole-dataset baseline.

11. Performance Optimization

Three indexes were added to support the query patterns the dashboard actually uses — date-based trend queries, customer joins, and location filtering — and EXPLAIN was used to inspect how MySQL resolves each one.

```
CREATE INDEX idx_sales_order_date ON Sales(order_date);
CREATE INDEX idx_sales_customer_id ON Sales(customer_id);
CREATE INDEX idx_customers_location ON Customers(location);
```

Index	Supports
idx_sales_order_date	Time-based reporting — the monthly trend view and sp_MonthlySummary
idx_sales_customer_id	Joins to Customers and customer-level KPI reporting
idx_customers_location	Regional filtering and vw_RegionalSummary

EXPLAIN was used to inspect these query paths, and aggregated reporting views were used wherever possible so the dashboard queries pre-shaped data instead of aggregating the full Sales table on every filter change. In the interest of honesty: this project does not claim that every single query always used a specific index in every execution — EXPLAIN was used for inspection and learning, not as a formal before/after benchmark (see Section 14 for what that follow-up work would look like).

12. Technical Challenges & Solutions

Challenge	Solution
CSV boolean import	Stored True/False source values as VARCHAR(5) at staging rather than casting immediately.
Tableau cross-filtering	Rebuilt worksheets around one unified “Sales+” SQL-backed data source.
Month labels for trend charts	Added a Tableau date calculation for clean month-level plotting.
Map geography	Mapped city-level locations to Indian states with a Tableau calculated field.
Radial satisfaction visual	Used a radial/circle-based visual instead of defaulting to another bar chart.
Lollipop chart construction	Built with a negative Lollipop-Stem calculation combined with Gantt and circle mark layers.
No genuine profit field	Reported revenue as revenue — did not fabricate or relabel a profit figure.

13. Lessons Learned

- Window functions solve ranking and period-comparison problems far more cleanly than self-joins or manual subtotal queries.
- CTEs make multi-stage business logic (like month-over-month growth) genuinely readable — worth the extra lines.
- Views create a stable contract between SQL and the BI layer, so a dashboard doesn't break just because someone edits a raw table.
- A live BI connection is only as good as the shared data model behind it — the “Sales+” join was the single biggest factor in getting filters to behave consistently.
- Mixing visual forms (radial, lollipop, heatmap, map) genuinely improves how quickly an executive can scan a dashboard, compared to an all-bar-chart layout.
- Performance claims should be measured and documented, not assumed — this is exactly why Section 11 is careful about what EXPLAIN did and didn't prove.
- Stating a dataset's limitations plainly (Section 3.4) is more useful to a reader than quietly working around them.

14. Limitations & Future Improvements

14.1 Current limitations

- One unique customer_id per order limits any repeat-customer or lifetime-value analysis.
- No genuine cost or profit field exists in the source data.
- The Tableau workbook is connected to a local MySQL instance, not a hosted/cloud database.
- Traffic and weather relationships shown on the dashboard are descriptive correlations, not causal claims.

14.2 Planned improvements

- Add repeat-customer history and lifetime-value tracking (requires a dataset with repeat visits).
- Add cost, discount, commission, and profit fields to enable true margin analysis.
- Move the database to a cloud data warehouse for shared, always-on access.
- Add scheduled refresh and monitoring instead of manual updates.
- Add row-level security so different stakeholders see only their permitted region/data.
- Capture formal before/after EXPLAIN plans as quantitative evidence of the indexing improvement, rather than qualitative inspection alone.

15. Final Deliverables & Submission Package

Deliverable	Status
SQL script (schema, views, procedure, indexes)	Completed
Tableau workbook (.twb)	Completed
Architecture diagram (PDF/PNG)	Completed
Portfolio case study (this document)	Completed
README	To be finalized on GitHub
Dashboard screenshots	To be packaged on GitHub
Published dashboard link	Pending — depends on Tableau edition/hosting
LinkedIn project post	Next submission step

Repository structure

```
Task6-Business-Analytics/  
├─ README.md  
├─ SQL/Task6_Business_Analytics_Final.sql  
├─ Tableau/Task6_Business_Analytics_Final.twb  
├─ Documentation/Task6_Business_Analytics_Portfolio_Case_Study_FINAL.pdf  
├─ Documentation/Task6_SQL_to_Dashboard_Architecture.pdf  
├─ Documentation/Task6_SQL_to_Dashboard_Architecture.png  
└─ Screenshots/
```

16. Conclusion

Task 6 took a raw, 20,000-row food-delivery export and turned it into a structured SQL-to-BI analytics solution: a normalized relational model, advanced window-function analytics, a reusable reporting layer, a parameterized stored procedure, targeted indexing, and a live-connected, multi-filter Tableau executive dashboard.

More importantly, the project was built to be honest about what the data can and cannot support — there is no fabricated profit figure, no invented repeat-customer story, and no performance claim beyond what EXPLAIN actually showed. That discipline is what makes the solution reproducible and genuinely portfolio-ready, rather than just visually polished.

— Subham Kumar

PROJECT	ORGANIZATION	SUBMISSION TYPE
Task 6 — SQL Business Analytics	MainCrafts Technology	Portfolio Case Study

Thank you for reading — full SQL scripts and the live Tableau workbook accompany this document in the submission package.